Laboratory Setup Guide

Lab lessons are a fundamental part of the Operating Systems course. The course encompasses 10 lab assignments that are designed to make you familiar with commands and system calls that can be found in any UNIX-compliant environment (as defined by the POSIX standard). In particular, during these lab lessons we will:

- Learn how to interact with the operating system via shell using the POSIX command line interface and utilities;
- Learn how to interact with the operating system via C code using the POSIX operating system API;
- Learn how to automate simple tasks by writing scripts using the BASH scripting language.

To work on the assignments you will need access to a suitable UNIX environment. In theory, any UNIX-compliant system with BASH (or a BASH-compliant shell), the GCC compiler and a text editor installed would work. However, we advise you to use a recent Linux distribution, like Ubuntu, Debian or Mint, as all of the POSIX functionalities that we will need during the assignments are guaranteed to be available on them. You can choose to use either a physical or virtualized system.

Setting up a UNIX environment on your physical machine

If you happen to already have Linux installed on your physical machine, you are almost good to go! Most Linux distributions use BASH as their default shell, just be sure to have GCC installed (which package to install depends on the distro you are using, checkout this useful guide on how to install the GCC compiler on linux system).

If you have a Mac you may be able to work on the assignements directly on your physical machine (with some minor tweaks). Since macOS is a UNIX operating system, the POSIX commands and API that we will use in this course are all available (with the exception of unnamed semaphores, for which there is however a drop-in replacement). Jump to the following subsection to see how to set up macOS to work on the assignments.

Depending on your hardware, you may have the option to configure a dual boot system by installing Linux on a separate disk or partition. Although setting up a dual boot system is a rather formative (and at times frustrating) experience, we do not recommend it for the sole purpose of working on the assignments of this course. If you decide to go down that route we advise you to take your time to backup your machine and learn about disk partitioning, bootloading and UEFI before any attempt (you are kind on your own on that, sorry).

Setting up macOS to work on the assignments

The default shell in macOS is zsh which is based on BASH, so you won't have any problems writing scripts using the BASH scripting language.

We are not so lucky when it comes to the compiler: macOS use clang as its default C compiler while in this course we will teach you to use GCC. You can use clang to compile your programs if you really want (nothing wrong with that) but in that case you will be on your own when it comes to solving any problem you may have. Luckily you can easily install GCC on you Mac, following these steps:

1. Install homebrew (if not already installed)
2. Install GCC: `brew install gcc`
3. Check the version of the installed gcc package: `brew list --versions gcc`. The output will be something like: `gcc 12.2.0`, which tells us that our installed version is version 12.
4. Check the location in which GCC is installed: `which gcc-<VERSION>`. E.g., `which gcc-12`. The output will be something like `/opt/homebrew/bin/gcc-12` on devices with Apple silicon or `/usr/local/bin/gcc-12` on devices with Intel chips.
5. (Optional) By default on macOS the `gcc` command points to a copy of the clang compiler (`/usr/bin/gcc`). If you want to invoke the GCC compiler we just installed with brew using the command `gcc` instead of `gcc-<VERSION>` you need to create a symlink to its executable in the same directory in which the GCC executable is installed.

```
cd $(dirname <GCC_LOCATION>)

sudo ln -s gcc-<VERSION> gcc
```

Once you have GCC installed, you are up and ready to work on the assignments of this course. The only issue you will have is that you will not be hable to use unnamed POSIX semaphores. In fact, although the macOs pthreads implementation complies with the POSIX standard, it does not include all the pthreads and associated functionality available on Linux. In particular, macOS's implementation of the pthread library does not support unnamed semaphores. A possible workaround is to use named semaphores instead on unnamed ones, using `sem_open()` and `sem_close()` instead of `sem_init()` and `sem_destroy()`. Have a look at this documentation page for an overview of the differences between the two.

Setting up a UNIX environment on a virtual machine

Perhaps the easiest way to set up a suitable UNIX environment to work on the assignments is to use a virtual machine. You have two options here: you can either use a self-hosted virtual machine or you can connect to a remote virtual machine running on one of Polito's virtualization servers.

If you prefer to use your own virtual machine, depending on your machine's native OS you have different options, refer to the appropriate section below. Otherwise, follow this guide to know how to connect to the remote virtual machine.

Setting up a virtual UNIX environment on Windows

If you want to host a Linux virtual machine on Windows you have the following options:

- Install a Linux virtual machine using VirtualBox.
- Use the Windows Subsystem for Linux version 2 (WSL 2) feature. Refer to the Microsoft documentation.
- Use a Docker container: refer to the following subsection.

Setting up a virtual UNIX environment on macOS

If you have a Mac but still prefer to use Linux for the assignments, you can set up a virtual environment in a few different ways:

- If you have a Mac with an Apple chip and happen to own a Parallels licence, you can install a Linux virtual machine using Parallels.
- Otherwise, if you have an Intel chip, you can install a Linux virtual machine using VirtualBox.
- An alternative approach that works in both cases is to use a Docker container, refer to the following subsection.

Setting up Docker to work on the assignments

To set up an Ubuntu Docker container follow these steps:

1. Install Docker Desktop: use this link if you are on Windows or this link if you are on macOs.
2. Get the official Ubuntu Docker image from Docker Hub:

```
docker pull ubuntu
```

3. Run Ubuntu in a container:

```
docker run --name os_ubuntu ubuntu
```

4. Step inside the container's shell:

```
docker exec -it os_ubuntu bash
```

5. You can use VSCode with the Remote Development extension pack to write programs inside the